

# COMP3411/COMP9814 Artificial Intelligence

## Assignment 1: Artificial Neural Networks

Term 3, 2025

**Due:** Friday, 17 October 2025, 5:00 PM AEST

**Worth:** 20 marks + 5 marks (tutorial participation) = 25% of final grade

**Submission:** Electronic submission via Moodle

### 1 Introduction

This assignment explores the application of artificial neural networks to climate prediction, specifically forecasting the El Niño-Southern Oscillation (ENSO) phenomenon. ENSO is one of the most important climate patterns affecting global weather, making its prediction crucial for agricultural planning, disaster preparedness, and resource management worldwide.

You will build and evaluate multiple neural network architectures to predict the Niño3.4 index, the standard metric for monitoring ENSO. Starting with simple single-output networks and progressing to sophisticated multi-output models, you will gain hands-on experience with neural network techniques, including transfer learning and hyperparameter optimisation.

## 2 Background

### 2.1 The ENSO Phenomenon

The El Niño–Southern Oscillation (ENSO) is a major driver of seasonal and year-to-year climate variability that shapes global weather systems. This coupled ocean–atmosphere process, characterised by irregular changes in wind patterns and sea surface temperatures (SST) across the tropical Pacific, has profound impacts on weather patterns worldwide [1, 2, 3]. ENSO cycles through three states: El Niño (warming), La Niña (cooling), and neutral conditions, occurring on scales of months to years.

The societal importance of ENSO cannot be overstated. It influences agricultural yields, water resources, and extreme weather events including droughts, floods, hurricanes, and heatwaves across the globe [4, 5, 6, 7]. Weather centres worldwide actively forecast ENSO as a critical component of their seasonal prediction systems, helping governments and industries prepare for its impacts [8].

### 2.2 The Niño3.4 Index

To monitor and quantify ENSO, researchers use numerical indices that capture different components of the system. The most widely applied is the Niño3.4 index, which measures the mean SST anomaly in the equatorial Pacific (5°N–5°S, 170°W–120°W), as shown in Figure 1. This index serves as the primary metric for determining ENSO phase and intensity.

## 2.3 The Neural Network Approach

Predicting the Niño3.4 index presents a challenging time-series forecasting problem due to the complex, non-linear interactions between ocean and atmosphere. Traditional physics-based models require enormous computational resources and still struggle with long-term predictions. Neural networks offer a promising alternative approach, capable of learning complex patterns from historical data without explicit physical equations.

In this assignment, you will develop neural network models to predict the Niño3.4 index for future months using past monthly values as input features.

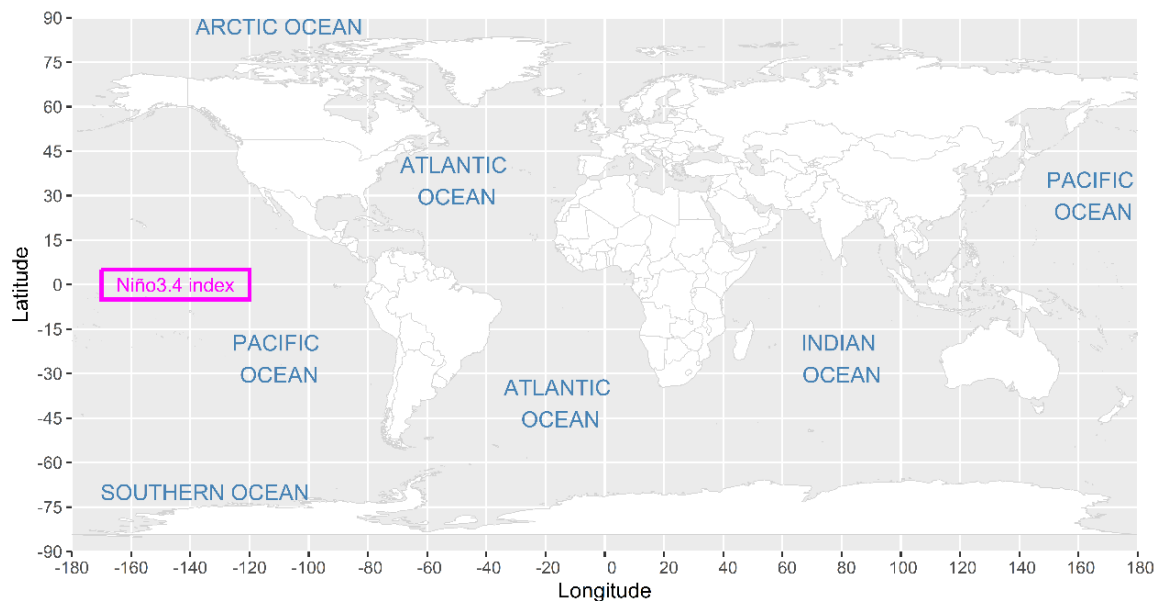


Figure 1: World map showing the region of the Niño3.4 index.

## 3 Assignment Specification

### 3.1 Data Description

The Niño3.4 index time-series data for current, previous, and future months is provided in `Nino3.4_data.csv` (see Table 1).

Env: pytorch      Colab → 连接 drive      To GPU  
连接 Nvidia      !nvidia-smi  
Coding style

Table 1: Structure of Nino3.4\_data.csv Dataset

| Column Name                               | Description                          | Time Reference |
|-------------------------------------------|--------------------------------------|----------------|
| <i>Temporal Identifiers</i>               |                                      |                |
| year                                      | Calendar year of observation         | —              |
| month                                     | Calendar month of observation (1-12) | —              |
| <i>Input Features (Historical Values)</i> |                                      |                |
| nino_t                                    | Niño3.4 index at current month       | t              |
| nino_tminus1                              | Niño3.4 index 1 month ago            | t-1            |
| nino_tminus2                              | Niño3.4 index 2 months ago           | t-2            |
| <i>Prediction Targets (Future Values)</i> |                                      |                |
| nino_tplus1                               | Niño3.4 index 1 month ahead          | t+1            |
| nino_tplus2                               | Niño3.4 index 2 months ahead         | t+2            |
| nino_tplus3                               | Niño3.4 index 3 months ahead         | t+3            |
| nino_tplus4                               | Niño3.4 index 4 months ahead         | t+4            |
| nino_tplus5                               | Niño3.4 index 5 months ahead         | t+5            |
| nino_tplus6                               | Niño3.4 index 6 months ahead         | t+6            |

### 3.2 Assignment Tasks

You are required to complete two main tasks:

#### 3.2.1 Task A - Multiple Individual Models

Build 6 separate neural networks, where each network specialises in predicting a single future time point. Network 1 predicts nino\_tplus1 (1 month ahead), Network 2 predicts nino\_tplus2 (2 months ahead), and so on through Network 6 which predicts nino\_tplus6 (6 months ahead). Each network should be trained independently with its own set of optimised hyperparameters.

#### 3.2.2 Task B - Single Multi-Output Model

Build a single unified neural network that simultaneously predicts all 6 future time points. This network will have 6 output neurons, producing predictions for nino\_tplus1, nino\_tplus2, through nino\_tplus6 in a single forward pass. This approach allows the model to learn shared representations across all forecast horizons.

### 3.3 Input Features

Both tasks use the same three input features: nino\_t (current month), nino\_tminus1 (1 month ago), and nino\_tminus2 (2 months ago). These three consecutive monthly values of the Niño3.4 index provide the temporal context needed for forecasting future values.

### 3.4 Evaluation Metrics

Your models will be evaluated using two primary metrics:

- **Root Mean Square Error (RMSE):** Measures the average magnitude of prediction errors. Lower values indicate better performance.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

where  $y_i$  is the actual value and  $\hat{y}_i$  is the predicted value.

- **Pearson Correlation Coefficient:** Measures the linear relationship between predictions and actual values. Values closer to 1 indicate stronger positive correlation.

$$r = \frac{\sum_{i=1}^n (y_i - \bar{y})(\hat{y}_i - \bar{\hat{y}})}{\sqrt{\sum_{i=1}^n (y_i - \bar{y})^2 \sum_{i=1}^n (\hat{y}_i - \bar{\hat{y}})^2}}$$

These metrics should be calculated for each forecast horizon to assess how model performance degrades with longer prediction windows.

## 4 Task A: Individual Neural Networks for Each Forecast Horizon

### 4.1 Data Preparation

- Load the data from `Nino3.4.data.csv`
- Read test years from `test_years.csv`. Extract all data rows from step (a) that match these test years to create your test set. This test data must remain untouched during training. You should use this test set to evaluate your model's performance and report results.

### 4.2 Neural Network Development

- Subset your test data based on the test years, then randomly partition the remaining dataset into training and validation sets. Choose an appropriate split ratio that ensures sufficient data for both training and validation.
- Pre-processing:** Apply scaling transformations to the input features (`nino_tminus2`, `nino_tminus1`, `nino_t`). Scaling the target variables (`nino_tplus1` through `nino_tplus6`) is optional - if you choose to scale them, save the target scalers for inverse transformation during evaluation. Apply the same transformations to the validation set.

*Optional: You might also consider creating additional features from the month column. For example, some students find it helpful to encode the cyclical nature of months using sine and cosine transformations:  $\sin(2\pi \times \text{month}/12)$  and  $\cos(2\pi \times \text{month}/12)$ . This can help the model understand that December is adjacent to January.*

**Note:** Save your scaler objects for later use (e.g., using `pickle` or `joblib`).

- Train a neural network that predicts the Niño3.4 index of the next month, i.e. `nino_tplus1` using reasonable hyperparameters. Ensure reproducibility by setting

a seed throughout your code. Plot the training and validation loss curves across epochs.

**Note:** Save your trained models in a format appropriate for your chosen framework. Common formats include `.keras`, `.h5`, `.pth`, `.pt`, or `.onnx`. Ensure your models can be loaded during evaluation.

- (f) Repeat (e) for the subsequent months, i.e., `nino_tplus2`, `nino_tplus3`,... `nino_tplus6`. Together with part (e), this will give you six trained networks and six plots for training and validation loss, as shown in Figure 2.

**Task A: Training History for Individual Models**

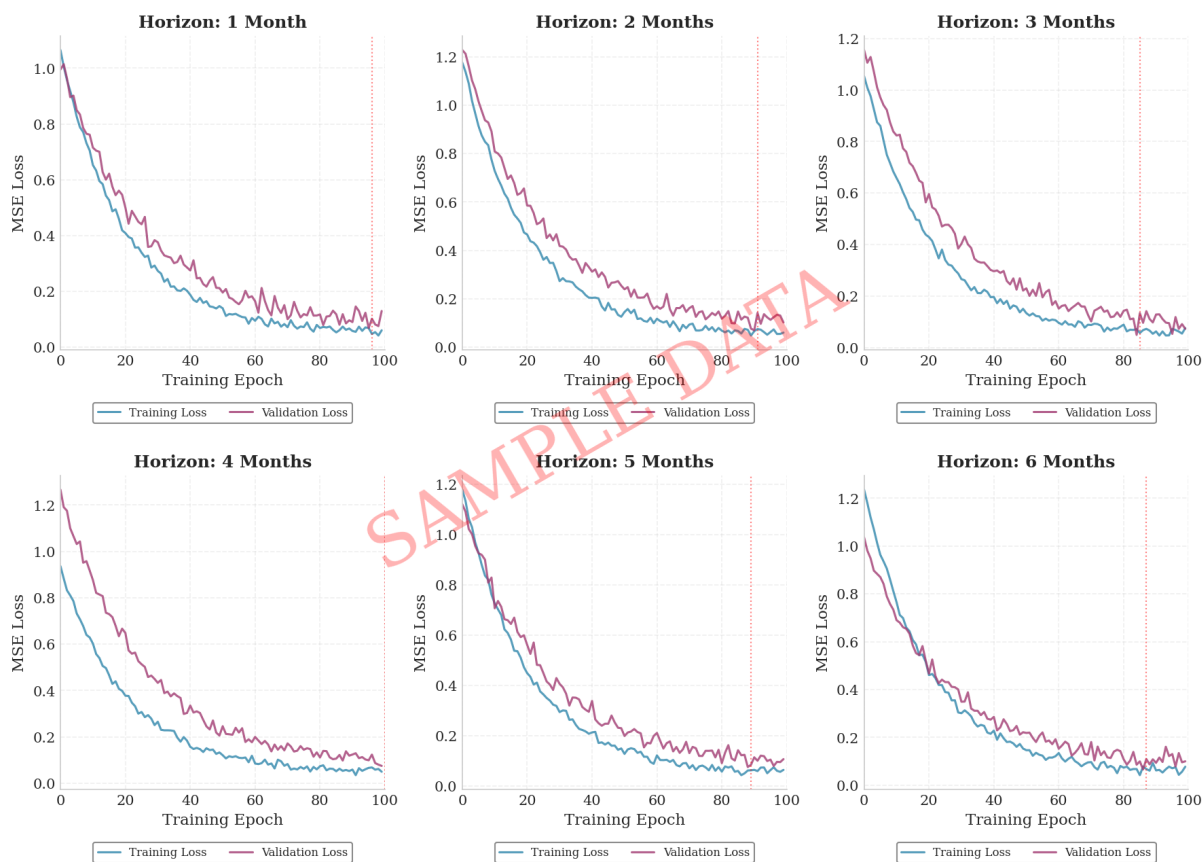


Figure 2: Example: Task A Training History for Individual Models (Sample Data)

### 4.3 Model Evaluation

- (j) Apply the same transformations to the test set as you did to the training and validation sets.
- (k) Use your models to forecast the Niño3.4 index on the test set for the months ahead they are designed for.
- (l) Evaluate each model by calculating the RMSE and Pearson Correlation Coefficient between the predicted and true values (as defined in Section 3.4).
- (m) Display results in a table showing RMSE and correlation coefficients for each forecast horizon (1-6 months ahead).

## 4.4 Model Improvements

- (n) Apply a strategy that involves transfer learning to the model that predicts `nino_tplus6` and calculate the RMSE and correlation coefficient on the testing data.

*Hint: First train a model on an easier task such as `nino_tplus1`, then continue training the same model to predict `nino_tplus6`.*

# 5 Task B: Multi-Output Neural Network

## 5.1 Neural Network Development

- (o) Subset your test data based on the test years, then randomly partition the remaining dataset into training and validation sets. Choose an appropriate split ratio that ensures sufficient data for both training and validation.
- (p) Pre-processing: Apply scaling transformations to the input features (`nino_tminus2`, `nino_tminus1`, `nino_t`). Scaling the target variables (`nino_tplus1` through `nino_tplus6`) is optional - if you choose to scale them, save the target scalers for inverse transformation during evaluation. Apply the same transformations to the validation set.

**Note:** Save your scaler objects for later use (e.g., using pickle or joblib).

- (q) **Hyperparameter Tuning:** Build and optimise a multi-output neural network that simultaneously predicts all six future months (`nino_tplus1` through `nino_tplus6`) in a single forward pass. Note that your output layer must have 6 neurons corresponding to these six predictions.

Start with a reasonable baseline architecture and systematically explore how changes in network depth, width, and learning rate affect performance to prevent overfitting.

- (r) **Training Visualisation:** Create a plot showing the training and validation loss curves for your best model.

**Note:** Save your trained model in a format appropriate for your chosen framework.

## 5.2 Model Evaluation

- (s) Calculate performance metrics on the testing data for each output (all 6 forecast horizons) using the metrics defined in Section 3.4.
- (t) Create scatter plots of predicted vs observed values with RMSE and correlation metrics for each forecast horizon, as shown in Figure 3.

## Task B: Multi-Output Model - Predictions vs Observations

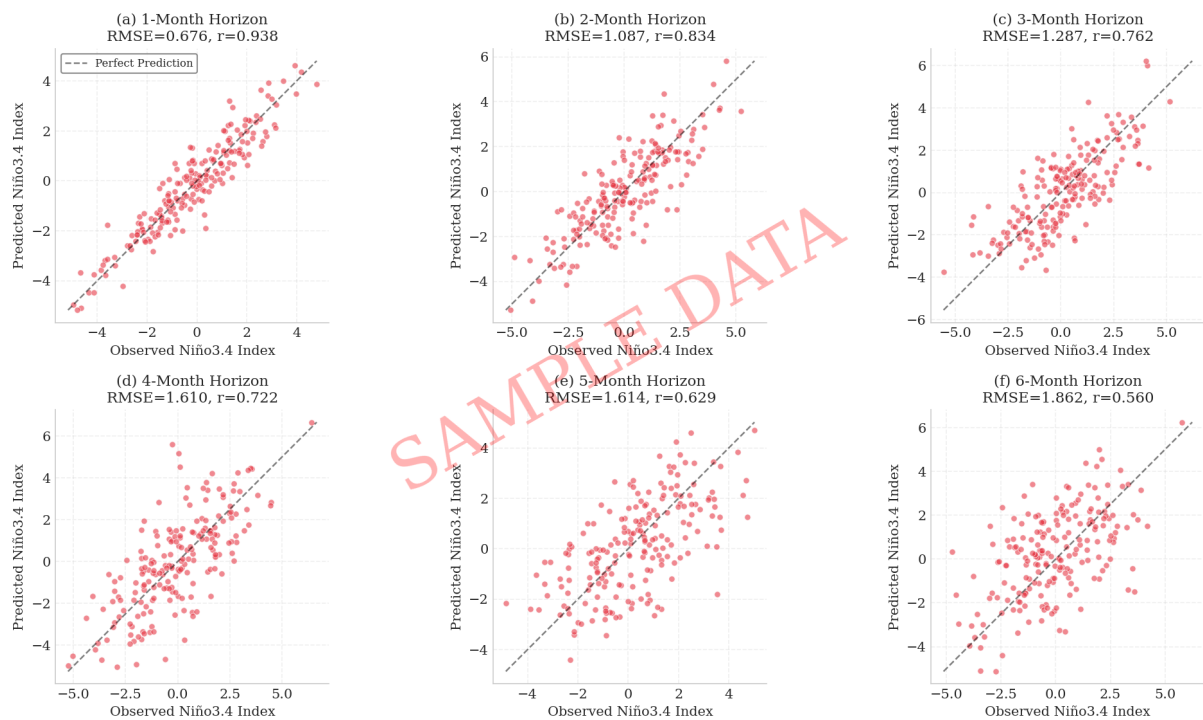


Figure 3: Example: Task B Scatter Plots - Predictions vs Observations (Sample Data)

### 5.3 Model Improvements

- (u) Propose and implement a strategy that could improve the performance of such a multi-output model.

*Hint: think about modifying the loss function so that it gives different weights to different outputs, or combining errors in a customised way instead of using the default mean squared error.*

**Note:** Save this improved model separately from your standard Task B model.

## 6 Comparison and Analysis

Now that you have implemented both approaches, it's time to analyse their strengths and weaknesses. The two tasks represent fundamentally different strategies for multi-step time-series forecasting. Task A uses individual models where each specialises in one specific forecast horizon, allowing it to learn patterns unique to that time scale. This specialisation can lead to better accuracy for individual horizons but requires training and maintaining multiple models. In contrast, Task B uses a single model that learns to predict all horizons simultaneously, potentially discovering shared patterns and relationships across different time scales. This approach is more efficient but may compromise accuracy for some horizons.

In your comparison, consider the following:

- **Performance degradation:** How does prediction accuracy change from  $t+1$  to  $t+6$ ? Which approach handles long-term predictions better?
- **Model complexity:** Did you need different architectures for different horizons in Task A? How complex did Task B's model need to be to handle all outputs?
- **Training behaviour:** Which models converged faster? Did near-term predictions stabilise before long-term ones?
- **Resource usage:** Compare training time, memory requirements, and inference speed between the two approaches.
- **Practical considerations:** Which approach would be easier to deploy and maintain in a production environment?

Your analysis should help you understand when each approach is most appropriate in real-world forecasting applications.

## 6.1 Comparison Requirements

- (w) Create a comparison table showing RMSE and correlation coefficients for all 6 forecast horizons, comparing: (1) Task A individual models, (2) Task B standard model (baseline), and (3) Task B improved model with weighted loss. Include average performance and calculate relative improvement percentages using Task B standard model as the baseline reference.
- (x) Generate comparison visualisations showing RMSE and correlation trends across forecast horizons for both approaches, as shown in Figure 4.

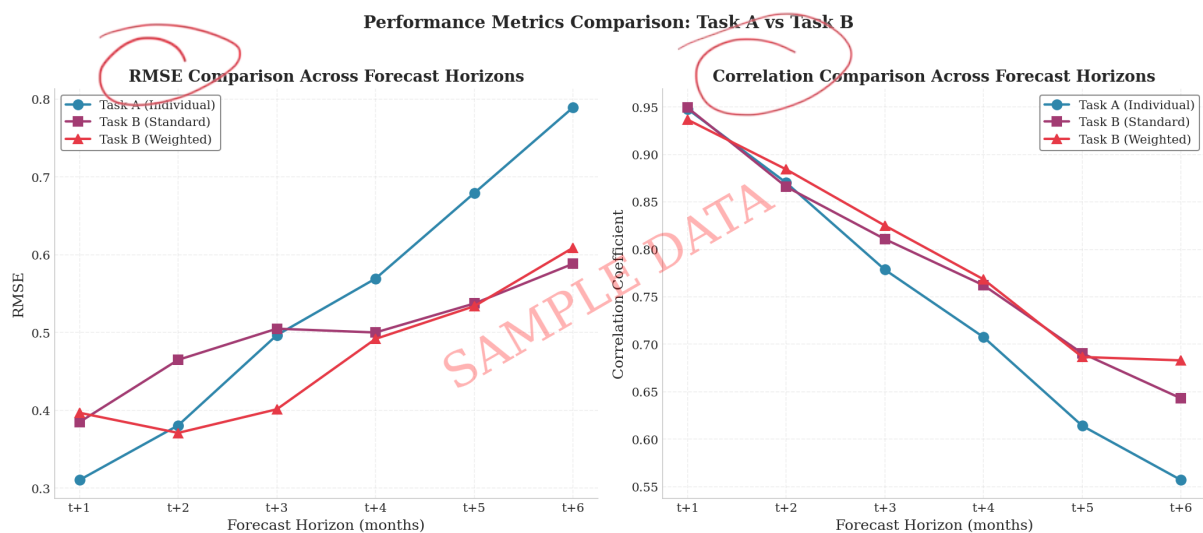


Figure 4: Example: Performance Metrics Comparison - Task A vs Task B (Sample Data)

## 7 Assessment and Evaluation

### 7.1 Evaluation

Your assignment will be evaluated during a scheduled discussion session with your tutor. Attendance is compulsory - submissions not discussed will receive zero marks.

Prior to the discussion session, you will be provided with a hidden test dataset (Nino3.4\_data\_hidden.csv) that has the same structure as the training data but contains data from different years. You must create your own evaluation script that loads this hidden dataset, applies your trained models, and generates evaluation metrics (See a sample code in Section 8).

During the discussion, you will run your evaluation script on the hidden test dataset, demonstrating that your models work correctly on unseen data. Be prepared to explain your design decisions and understand the trade-offs between individual models (Task A) and multi-output models (Task B).

### 7.2 Assessment Breakdown

Table 2: Assignment Assessment Breakdown

| Assessment Component                                                     | Marks     | %                         |
|--------------------------------------------------------------------------|-----------|---------------------------|
| <b>Assignment (20 marks)</b>                                             |           |                           |
| <i>Implementation Components (8 marks, 40%)</i>                          |           |                           |
| Task A: Individual networks for 6 forecast horizons (t+1 to t+6)         | 2 ✓       | 10%                       |
| Task A: Transfer learning from t+1 to t+6 models                         | 1 ✓       | 5%                        |
| Task B: Single multi-output network for all 6 horizons                   | 2 ✓       | 10%                       |
| Task B: Improved model with weighted loss function                       | 1 ✓       | 5%                        |
| Visualisations and performance metrics                                   | 1 ✓       | 5%                        |
| Code quality and submission requirements                                 | 1 ✓       | 5%                        |
| <i>Understanding &amp; Discussion (12 marks, 60%)</i>                    |           |                           |
| Architecture design: layers, neurons, activation functions               | 3 ✓       | 15%                       |
| Neural network concepts: loss functions, optimisers, training techniques | 3 ✓       | 15%                       |
| Results analysis: RMSE/correlation trends across horizons                | 2         | 10%                       |
| Individual vs multi-output model comparison                              | 2         | 10%                       |
| Transfer learning strategy and implementation                            | 1         | 5%                        |
| Model limitations and improvement suggestions                            | 1         | 5%                        |
| <b>Assignment Total</b>                                                  | <b>20</b> | <b>100%</b>               |
| <b>Tutorial Participation (5 marks)</b>                                  |           |                           |
| Practical work and engagement in tutorials                               | 5         | —                         |
| <b>Overall Total</b>                                                     | <b>25</b> | <b>25% of final grade</b> |

## 8 Submission Requirements

Since your assignment will be evaluated in a discussion session, you must prepare your submission to facilitate efficient evaluation without running code during the session.

Submit the following files in a single ZIP archive named `zID_assignment1.zip`:

1. **Jupyter Notebook** (`zID_assignment1.ipynb`): Must show ALL output cells executed with all plots and results visible.
2. **Trained Models and Scalers**: Include all your trained models and preprocessing scalers. You must provide a total of 9 models: six individual Task A models for each forecast horizon (`tplus1` through `tplus6`), one Task A transfer learning model for `tplus6`, one Task B standard multi-output model, and one Task B improved model with weighted loss. Also include all scaler objects used for data preprocessing.
3. **Evaluation Cell in Notebook**: Your Jupyter notebook must include a dedicated evaluation cell (or cells) at the end that can load the hidden test dataset (`Nino3.4_data_hidden.csv`), load all your trained models and scalers, and evaluate them on this hidden dataset. This cell should print clear results showing RMSE and correlation metrics for all Task A and Task B models. During the discussion session, you will update the dataset path to the hidden test file and run this evaluation cell to demonstrate your models' performance on unseen data.

**Note:** Below is a sample structure for your evaluation cell. Please adapt this code to match your specific implementation, framework, and file naming conventions.

Listing 1: Sample Evaluation Cell Structure

```
1 # Example evaluation structure (adapt to your implementation)
2 hidden_data = pd.read_csv("Nino3.4_data_hidden.csv")
3
4 # Load your scalers and apply preprocessing
5 # ... your code here ...
6
7 print("EVALUATION RESULTS")
8 print("-" * 40)
9
10 # Evaluate Task A models
11 for horizon in ['tplus1', 'tplus2', 'tplus3', 'tplus4', 'tplus5',
12                'tplus6']:
13     # Load model and evaluate
14     # Print: f"{horizon}: RMSE = {rmse:.3f}, Corr = {corr:.3f}"
15
16 # Evaluate Task B models
17 # ... your evaluation code ...
```

**Important:** If your models fail to load or run during the evaluation, you may lose up to 50% of the marks for that component. Test thoroughly before submission!

You can submit as many times as you like before the deadline—later submissions overwrite earlier ones. After submitting your file, a good practice is to take a screenshot of it for future reference.

Late submission penalty: UNSW has a standard late submission penalty of 5% per day from your mark, capped at five days from the assessment deadline; after that students

cannot submit the assignment.

## 9 Deadline and Questions

Deadline: Week 5, Friday 17 October 2025, 5pm. Please use the forum on Moodle to ask questions related to the assignment. We will prioritise questions asked in the forum. However, you should not share your code to avoid making it public and possible plagiarism. If that's the case, use the course email [cs3411@cse.unsw.edu.au](mailto:cs3411@cse.unsw.edu.au) as alternative.

Although we try to answer questions as quickly as possible, we might take up to 1 or 2 business days to reply, therefore, last-moment questions might not be answered timely.

For any questions regarding the discussion sessions, please contact directly your tutor. You can have access to your tutor email address through Table 3.

## 10 Tutor Information

Table 3: Tutor Contact Information

| No. | Class ID(s)                | Tutor            | Email                                                                                          |
|-----|----------------------------|------------------|------------------------------------------------------------------------------------------------|
| 1   | 4202, 6348, 13078, 13079   | Abhishek Pradeep | <a href="mailto:abhishek.pradeep@student.unsw.edu.au">abhishek.pradeep@student.unsw.edu.au</a> |
| 2   | 13192, 13193, 13198, 13199 | Adam Stucci      | <a href="mailto:a.stucci@unsw.edu.au">a.stucci@unsw.edu.au</a>                                 |
| 3   | 4198, 4204, 6344, 6350     | Hadha Afrisal    | <a href="mailto:hadha.afrisal@unsw.edu.au">hadha.afrisal@unsw.edu.au</a>                       |
| 4   | 4212, 4215, 6358, 6361     | Haitao Gao       | <a href="mailto:haitao.gao@student.unsw.edu.au">haitao.gao@student.unsw.edu.au</a>             |
| 5   | 4205, 4214, 6351, 6360     | Ishan Dubey      | <a href="mailto:i.dubey@student.unsw.edu.au">i.dubey@student.unsw.edu.au</a>                   |
| 6   | 4197, 4213, 6343, 6359     | Joffrey Ji       | <a href="mailto:joffrey.ji@student.unsw.edu.au">joffrey.ji@student.unsw.edu.au</a>             |
| 7   | 4199, 4209, 6345, 6355     | John Chen        | <a href="mailto:xin.chen9@student.unsw.edu.au">xin.chen9@student.unsw.edu.au</a>               |
| 8   | 4219, 6365, 13076, 13077   | Jonas Macken     | <a href="mailto:j.macken@student.unsw.edu.au">j.macken@student.unsw.edu.au</a>                 |
| 9   | 4206, 4216, 6352, 6362     | Leman Kirme      | <a href="mailto:l.kirme@unsw.edu.au">l.kirme@unsw.edu.au</a>                                   |
| 10  | 4201, 4211, 6347, 6357     | Maher Mesto      | <a href="mailto:m.mesto@unsw.edu.au">m.mesto@unsw.edu.au</a>                                   |
| 11  | 4223, 4224, 6369, 6370     | Marium Malik     | <a href="mailto:marium.malik@unsw.edu.au">marium.malik@unsw.edu.au</a>                         |
| 12  | 4203, 4208, 6349, 6354     | Peter Ho         | <a href="mailto:peter.ho2@student.unsw.edu.au">peter.ho2@student.unsw.edu.au</a>               |
| 13  | 4210, 4220, 6356, 6366     | Trishika Abrol   | <a href="mailto:t.abrol@student.unsw.edu.au">t.abrol@student.unsw.edu.au</a>                   |
| 14  | 4200, 4207, 6346, 6353     | Xiongyu Xie      | <a href="mailto:xiongyu.xie@student.unsw.edu.au">xiongyu.xie@student.unsw.edu.au</a>           |
| 15  | 4217, 4218, 6363, 6364     | Yixin Kang       | <a href="mailto:yixin.kang@student.unsw.edu.au">yixin.kang@student.unsw.edu.au</a>             |
| 16  | 4221, 4222, 6367, 6368     | Zahra Donyavi    | <a href="mailto:z.donyavi@unsw.edu.au">z.donyavi@unsw.edu.au</a>                               |

## 11 Academic Integrity

### Academic Integrity

This assignment must be completed individually. Do not share your code or solutions with other students, and do not post assignment content on external forums.

**Permitted:**

- Discussion of concepts and approaches with other students
- Consultation with course staff
- Use of course materials and textbooks
- AI tools for learning concepts and debugging (not complete solutions)

**Not Permitted:**

- Sharing or copying code between students
- Submitting code written by others
- Posting assignment content online

## References

- [1] J. Bjerknes. Atmospheric teleconnections from the equatorial Pacific. *Monthly Weather Review*, 97:163–172, 1969.
- [2] D. J. Karoly. Southern hemisphere circulation features associated with El Niño–Southern Oscillation events. *Journal of Climate*, 2:1239–1252, 1989.
- [3] M. A. Alexander et al. The atmospheric bridge: The influence of ENSO teleconnections on air–sea interaction over the global oceans. *Journal of Climate*, 15:2205–2231, 2002.
- [4] B. Lyon and A. G. Barnston. ENSO and the spatial extent of interannual precipitation extremes in tropical land areas. *Journal of Climate*, 18:5095–5109, 2005.
- [5] S. M. Vicente-Serrano et al. A multiscalar global evaluation of the impact of ENSO on droughts. *Journal of Geophysical Research: Atmospheres*, 116, 2011.
- [6] F. H. S. Chiew and T. A. McMahon. Global ENSO-streamflow teleconnection, stream-flow forecasting and interannual variability. *Hydrological Sciences Journal*, 47:505–522, 2002.
- [7] Sanaa Hobeichi et al. How well do climate modes explain precipitation variability? *npj Climate and Atmospheric Science*, 7(1):295, 2024.
- [8] B. P. Kirtman et al. Current status of ENSO forecast skill: A report to the CLIVAR Working Group on Seasonal to Interannual Prediction. Technical report, CLIVAR Working Group, 2001.